

# ETL Recovery & Resilient Data Processing System

## Executive Summary

**Incident:** ETL job crashed processing millions of records for 3,500 app IDs from ADB → MCC database  
**Root Cause:** Single monolithic ETL job overwhelmed by high data volume  
**Solution:** Partition-based Spring Boot microservice with monitoring and auto-recovery  
**Timeline:** 2-day implementation (recovery + permanent solution)

## Table of Contents

- 1. Incident Analysis
- 2. Immediate Recovery Plan (Day 1)
- 3. Long-Term Solution Architecture
- 4. Implementation Plan
- 5. Monitoring & Alerting
- 6. Interview Preparation Guide
- 7. Technical Deep Dive

## Incident Analysis

### Problem Statement

Scenario:

- ETL job processing data from ADB database → MCC database
- 3,500 unique app\_ids need processing
- Millions of records (estimated 10-50 million based on 3,500 apps)
- Job crashed due to memory/timeout issues
- Daily volume: Normally low (thousands), but spikes can occur

Critical Questions:

- ☒ Why did it crash? Memory exhaustion, timeout, database connection pool exhaustion?
- ☒ How much data per app\_id? Some app\_ids might have 100K+ records
- ☒ Can we process incrementally? Yes - partition by app\_id
- ☒ What's the acceptable recovery time? 2-4 hours for backlog, <1 hour for daily

### Root Cause Analysis (5 Whys)

**Why did the ETL job fail?** → It crashed processing millions of records

**Why did it crash?** → Single job tried to load all 3,500 app\_ids at once, causing memory overflow

**Why didn't it handle large volumes?** → No pagination/partitioning strategy, loaded all data into memory

**Why was there no monitoring?** → Monolithic batch job with no progress tracking or alerting

**Why no recovery mechanism?** → Job designed for "happy path" only, no failure handling

**Root Cause: Lack of partition-based processing, monitoring, and resilience**

---

## Immediate Recovery Plan (Day 1)

Phase 1: Emergency Data Recovery (2-4 hours)

### Step 1: Assess Current State (30 minutes)

```
-- Check how many app_ids have been processed
SELECT COUNT(DISTINCT app_id) as processed_apps
FROM mcc.target_table
WHERE last_updated >= '2026-01-14'; -- Yesterday's date

-- Find remaining app_ids
SELECT app_id, COUNT(*) as record_count
FROM adb.source_table
WHERE app_id NOT IN (
    SELECT DISTINCT app_id FROM mcc.target_table
    WHERE last_updated >= '2026-01-14'
)
GROUP BY app_id
ORDER BY record_count DESC;

-- Total records to process
SELECT COUNT(*) as remaining_records
FROM adb.source_table
WHERE app_id NOT IN (
    SELECT DISTINCT app_id FROM mcc.target_table
    WHERE last_updated >= '2026-01-14'
);
```

### Expected Results:

- Total app\_ids: 3,500
- Processed before crash: ~200-500 (estimate)
- Remaining: ~3,000-3,300
- Records per app\_id: 1,000 - 100,000 (varies widely)

### Step 2: Create Recovery Partition Table (15 minutes)

```
-- Create partition tracking table
CREATE TABLE mcc.etl_recovery_partitions (
    partition_id INT PRIMARY KEY IDENTITY(1,1),
    app_id VARCHAR(50) NOT NULL,
```

```

    record_count BIGINT,
    status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, IN_PROGRESS, COMPLETED,
    FAILED
    worker_id INT NULL,
    started_at DATETIME NULL,
    completed_at DATETIME NULL,
    error_message VARCHAR(MAX) NULL,
    retry_count INT DEFAULT 0
);

-- Insert all app_ids as partitions
INSERT INTO mcc.etl_recovery_partitions (app_id, record_count)
SELECT app_id, COUNT(*) as record_count
FROM adb.source_table
WHERE app_id NOT IN (
    SELECT DISTINCT app_id FROM mcc.target_table
    WHERE last_updated >= '2026-01-14'
)
GROUP BY app_id;

-- Create index for fast claiming
CREATE INDEX idx_status_worker ON mcc.etl_recovery_partitions(status, worker_id);

```

### Step 3: Quick Recovery Script (PowerShell + SQL) (1 hour to run)

```

# emergency-recovery.ps1
# Process app_ids in parallel (10 workers)

$workers = 1..10
$jobs = $workers | ForEach-Object {
    $workerId = $_
    Start-Job -Name "Worker-$workerId" -ScriptBlock {
        param($workerId)

        # Keep claiming and processing app_ids
        while ($true) {
            # Claim next partition (thread-safe)
            $result = Invoke-Sqlcmd -Query @"
                WITH NextPartition AS (
                    SELECT TOP 1 partition_id, app_id
                    FROM mcc.etl_recovery_partitions WITH (UPDLOCK, ROWLOCK)
                    WHERE status = 'PENDING' OR (status = 'FAILED' AND retry_count
< 3)

                    ORDER BY record_count ASC -- Process small partitions first
                )
                UPDATE p
                SET status = 'IN_PROGRESS',
                    worker_id = $workerId,
                    started_at = GETDATE(),
                    retry_count = retry_count + 1
                OUTPUT INSERTED.partition_id, INSERTED.app_id
            "
        }
    }
}

```

```

        FROM mcc.etl_recovery_partitions p
        INNER JOIN NextPartition np ON p.partition_id = np.partition_id
"@

    if ($result -eq $null) {
        Write-Host "Worker $workerId: No more partitions"
        break
    }

    $appId = $result.app_id
    Write-Host "Worker $workerId processing app_id: $appId"

    try {
        # Transfer data for this app_id
        Invoke-Sqlcmd -Query @"
            INSERT INTO mcc.target_table (id, app_id, data_field,
last_updated)

            SELECT id, app_id, data_field, GETDATE()
            FROM adb.source_table
            WHERE app_id = '$appId'
"@

        # Mark completed
        Invoke-Sqlcmd -Query @"
            UPDATE mcc.etl_recovery_partitions
            SET status = 'COMPLETED', completed_at = GETDATE()
            WHERE partition_id = $($result.partition_id)
"@

    } catch {
        # Mark failed
        Invoke-Sqlcmd -Query @"
            UPDATE mcc.etl_recovery_partitions
            SET status = 'FAILED', error_message =
'$($_.Exception.Message)'
            WHERE partition_id = $($result.partition_id)
"@
    }
} -ArgumentList $workerId
}

# Monitor progress
while ((Get-Job -State Running).Count -gt 0) {
    $completed = Invoke-Sqlcmd -Query "SELECT COUNT(*) as cnt FROM
mcc.etl_recovery_partitions WHERE status='COMPLETED'"
    $failed = Invoke-Sqlcmd -Query "SELECT COUNT(*) as cnt FROM
mcc.etl_recovery_partitions WHERE status='FAILED'"
    $pending = Invoke-Sqlcmd -Query "SELECT COUNT(*) as cnt FROM
mcc.etl_recovery_partitions WHERE status='PENDING'"

    Write-Host "Progress: Completed=$($completed.cnt), Failed=$($failed.cnt),
Pending=$($pending.cnt)"
    Start-Sleep -Seconds 30
}

```

```
}

Get-Job | Receive-Job
Get-Job | Remove-Job
```

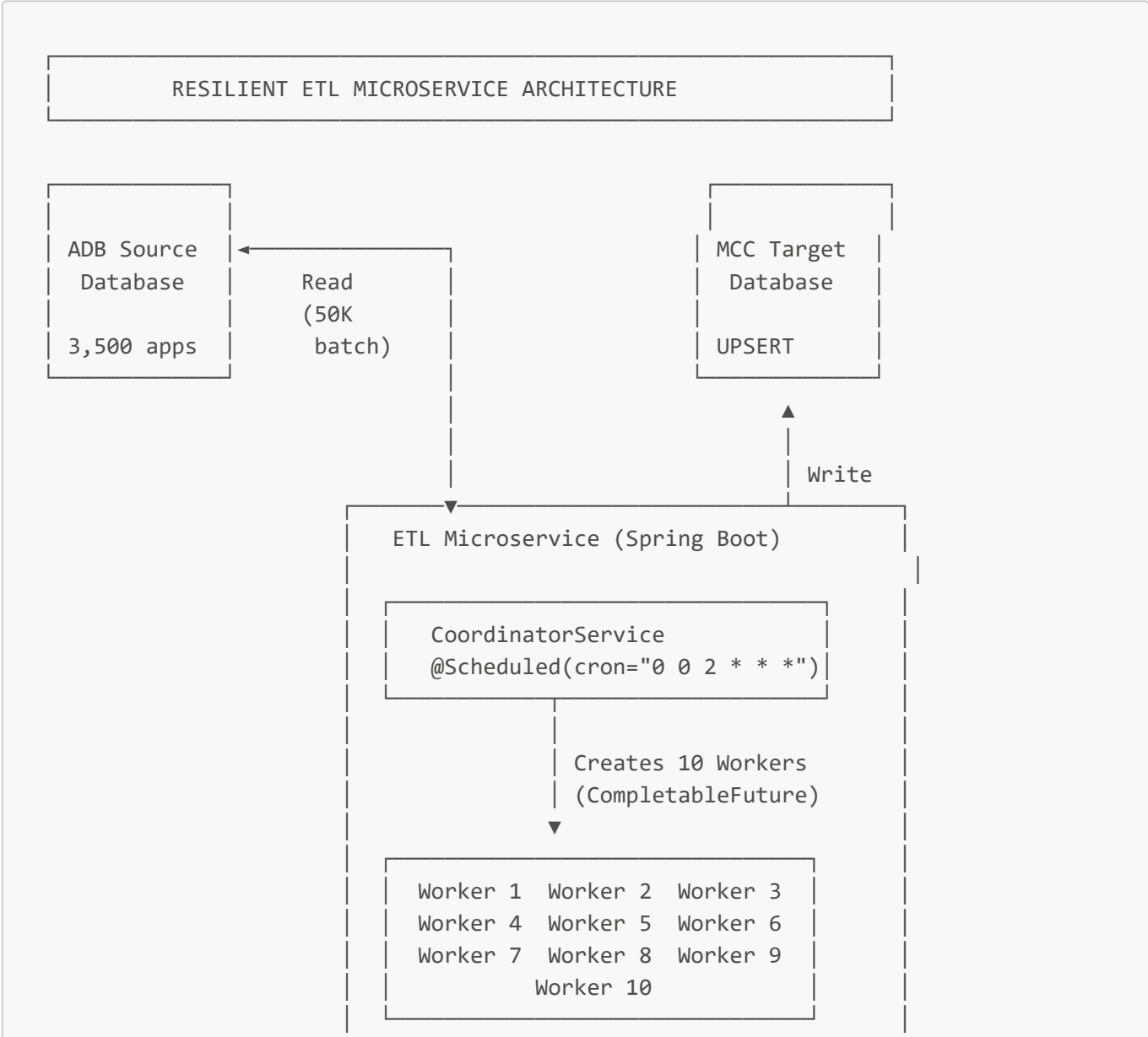
**Expected Recovery Time:** 2-4 hours for 3,500 app\_ids (depends on record volume)

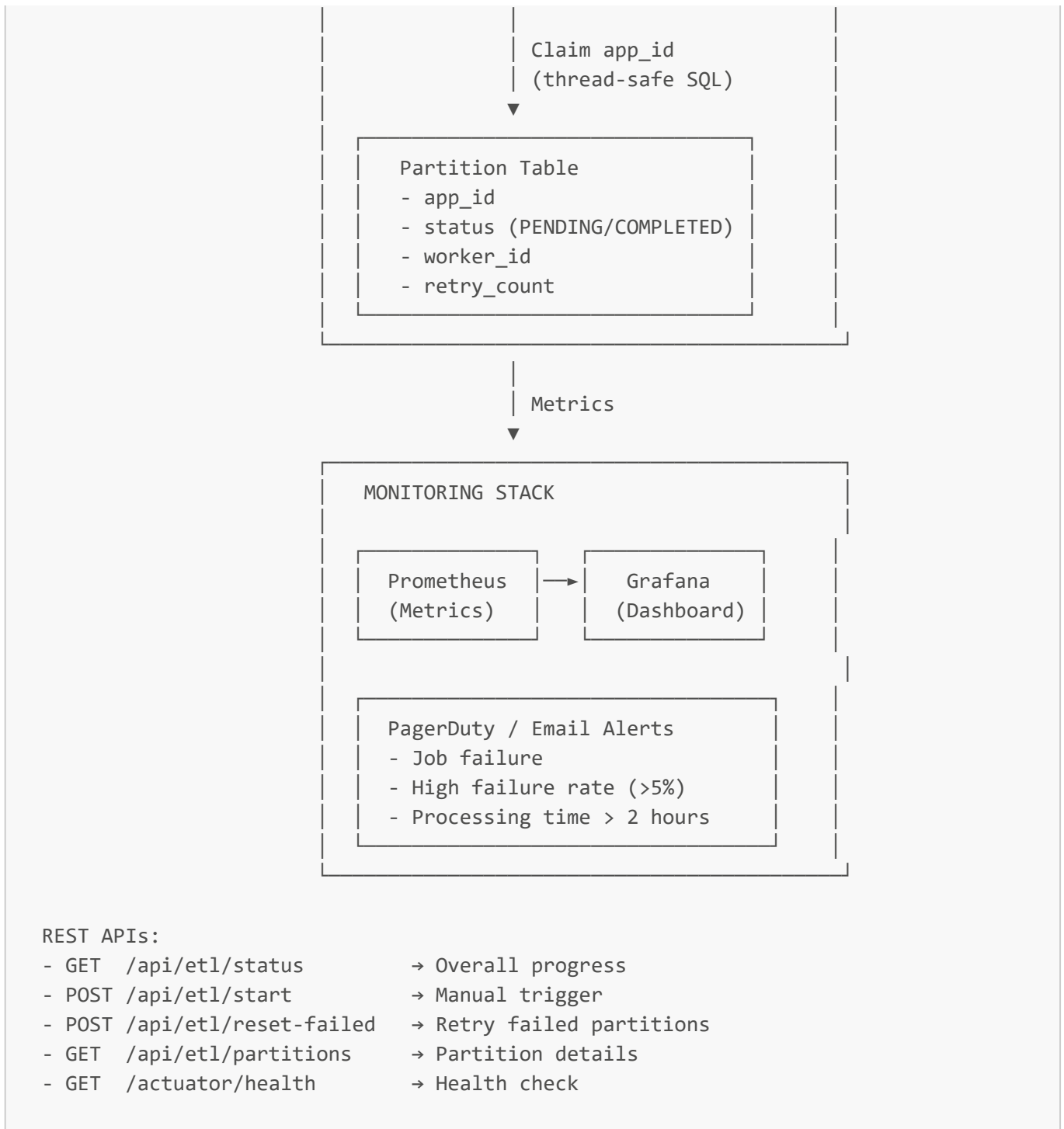
## Long-Term Solution Architecture

### Design Principles

- 1. **Partition by app\_id:** Each app\_id is independent unit of work
- 2. **Parallel processing:** 10 workers process concurrently
- 3. **Fault tolerance:** Failed partitions auto-retry, don't block others
- 4. **Monitoring:** Real-time progress tracking and alerting
- 5. **Idempotent:** Safe to rerun without duplicates

### Architecture Diagram





## Data Flow

### Daily Processing (Normal Volume)

1. Cron triggers at 2 AM daily
2. Query ADB: Find app\_ids with new/updated data (typically 10-100 apps)
3. Create partitions: Insert into etl\_partitions table
4. Start 10 workers: Each claims 1 app\_id at a time
5. Process partition:
  - a. Fetch records from ADB (batch size: 50,000)
  - b. Transform if needed
  - c. UPSERT into MCC database
  - d. Mark partition COMPLETED

6. All workers complete → Job SUCCESS
7. Duration: 5-15 minutes for normal daily volume

## High Volume Processing (3,500 apps scenario)

1. Manual trigger: POST /api/etl/start
2. Query ADB: Find all 3,500 app\_ids
3. Create 3,500 partitions
4. Process with priority:
  - Small app\_ids first (< 10K records) → Fast wins
  - Large app\_ids distributed across workers
5. If partition fails:
  - Mark FAILED with error message
  - Continue with other partitions
  - Auto-retry up to 3 times
6. Monitor progress:
  - Grafana dashboard shows real-time completion %
  - Alert if failure rate > 5%
7. Duration: 2-4 hours for 3,500 apps (10-50M records)

## Implementation Plan

### Phase 1: Database Setup (30 minutes)

#### MCC Database Tables

```
-- Partition tracking table
CREATE TABLE mcc.etl_partitions (
  partition_id BIGINT PRIMARY KEY IDENTITY(1,1),
  app_id VARCHAR(50) NOT NULL,
  batch_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
  record_count BIGINT DEFAULT 0,
  status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, IN_PROGRESS, COMPLETED,
  FAILED
  worker_id INT NULL,
  started_at DATETIME NULL,
  completed_at DATETIME NULL,
  error_message VARCHAR(MAX) NULL,
  retry_count INT DEFAULT 0,
  processing_time_ms BIGINT NULL
);

CREATE INDEX idx_status_worker ON mcc.etl_partitions(status, worker_id);
CREATE INDEX idx_batch_date ON mcc.etl_partitions(batch_date);

-- Job execution history
CREATE TABLE mcc.etl_jobs (
```

```

    job_id BIGINT PRIMARY KEY IDENTITY(1,1),
    job_type VARCHAR(50) NOT NULL, -- DAILY, MANUAL, RECOVERY
    started_at DATETIME NOT NULL,
    completed_at DATETIME NULL,
    total_partitions INT DEFAULT 0,
    completed_partitions INT DEFAULT 0,
    failed_partitions INT DEFAULT 0,
    total_records_processed BIGINT DEFAULT 0,
    status VARCHAR(20) DEFAULT 'RUNNING', -- RUNNING, COMPLETED, FAILED
    triggered_by VARCHAR(100) DEFAULT 'SCHEDULER'
);

-- Metrics table for monitoring
CREATE TABLE mcc.etl_metrics (
    metric_id BIGINT PRIMARY KEY IDENTITY(1,1),
    job_id BIGINT NOT NULL,
    metric_name VARCHAR(100) NOT NULL,
    metric_value DECIMAL(18,2) NOT NULL,
    recorded_at DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (job_id) REFERENCES mcc.etl_jobs(job_id)
);

-- Failed records (DLQ - Dead Letter Queue)
CREATE TABLE mcc.etl_failed_records (
    failed_id BIGINT PRIMARY KEY IDENTITY(1,1),
    partition_id BIGINT NOT NULL,
    app_id VARCHAR(50) NOT NULL,
    source_record_id BIGINT,
    error_message VARCHAR(MAX),
    failed_at DATETIME DEFAULT GETDATE(),
    retry_count INT DEFAULT 0,
    FOREIGN KEY (partition_id) REFERENCES mcc.etl_partitions(partition_id)
);

```

## Phase 2: Spring Boot Application Structure (1 day)

### Project Structure

```

etl-microservice/
├── pom.xml
├── src/main/java/com/example/etl/
│   ├── EtlApplication.java           # Main entry point
│   ├── config/
│   │   ├── DataSourceConfig.java     # ADB + MCC DataSources
│   │   ├── ExecutorConfig.java       # Thread pool configuration
│   │   └── SchedulerConfig.java       # @EnableScheduling
│   ├── controller/
│   │   ├── EtlController.java        # Manual trigger API
│   │   └── MonitorController.java     # Status/metrics API
│   ├── service/
│   │   └── CoordinatorService.java    # Main orchestration

```



```

├── PartitionService.java      # Partition management
├── WorkerService.java        # Process single partition
├── MetricsService.java        # Prometheus metrics
├── model/
│   ├── Partition.java         # Partition entity
│   ├── EtlJob.java            # Job entity
│   └── SourceRecord.java       # ADB record
├── repository/
│   ├── PartitionRepository.java # Partition CRUD
│   └── JobRepository.java       # Job CRUD
├── src/main/resources/
│   ├── application.properties # Database configs
│   └── logback-spring.xml      # Logging config

```

## Key Configuration Files

### pom.xml

```

<dependencies>
  <!-- Spring Boot Starter -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JDBC -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>

  <!-- SQL Server Driver -->
  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <version>12.4.2.jre11</version>
  </dependency>

  <!-- Monitoring -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>

  <dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
  </dependency>

  <!-- Lombok -->
  <dependency>

```

```

        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
    </dependency>
</dependencies>

```

## application.properties

```

spring.application.name=etl-microservice
server.port=8080

# ADB Source Database (Read-Only)
adb.datasource.url=jdbc:sqlserver://adb-server:1433;databaseName=ADB;encrypt=true
adb.datasource.username=adb_reader
adb.datasource.password=${ADB_PASSWORD}
adb.datasource.hikari.maximum-pool-size=20
adb.datasource.hikari.connection-timeout=30000
adb.datasource.hikari.idle-timeout=600000

# MCC Target Database (Read-Write)
mcc.datasource.url=jdbc:sqlserver://mcc-server:1433;databaseName=MCC;encrypt=true
mcc.datasource.username=mcc_writer
mcc.datasource.password=${MCC_PASSWORD}
mcc.datasource.hikari.maximum-pool-size=30
mcc.datasource.hikari.connection-timeout=30000

# ETL Configuration
etl.cron.schedule=0 0 2 * * *
etl.worker.count=10
etl.batch.size=50000
etl.retry.max-attempts=3
etl.partition.priority=record_count_asc # Process small partitions first

# Monitoring
management.endpoints.web.exposure.include=health,prometheus,metrics
management.metrics.export.prometheus.enabled=true
management.endpoint.health.show-details=always

# Logging
logging.level.com.example.etl=INFO
logging.file.name=logs/etl-microservice.log

```

## Phase 3: Core Service Implementation

### CoordinatorService.java (Main Orchestration)

```

@Service
@Slf4j
public class CoordinatorService {

```

```

@Autowired
private PartitionService partitionService;

@Autowired
private WorkerService workerService;

@Autowired
private MetricsService metricsService;

@Autowired
@Qualifier("mccJdbcTemplate")
private JdbcTemplate mccJdbc;

@Value("${etl.worker.count:10}")
private int workerCount;

// Scheduled daily run at 2 AM
@Scheduled(cron = "${etl.cron.schedule}")
public void scheduledEtlJob() {
    log.info("Starting scheduled ETL job");
    executeEtl("DAILY", "SCHEDULER");
}

// Manual trigger via REST API
public Long manualTriggerEtl(String triggeredBy) {
    log.info("Manual ETL trigger by: {}", triggeredBy);
    return executeEtl("MANUAL", triggeredBy);
}

public Long executeEtl(String jobType, String triggeredBy) {
    // Step 1: Create job record
    Long jobId = createJobRecord(jobType, triggeredBy);

    try {
        // Step 2: Identify app_ids to process
        List<String> appIds = identifyAppIdsToProcess(jobType);
        log.info("Job {}: Found {} app_ids to process", jobId, appIds.size());

        if (appIds.isEmpty()) {
            log.info("No app_ids to process");
            completeJob(jobId, 0, 0, 0);
            return jobId;
        }

        // Step 3: Create partitions
        List<Long> partitionIds = partitionService.createPartitions(jobId,
appIds);
        log.info("Job {}: Created {} partitions", jobId, partitionIds.size());

        // Step 4: Process partitions in parallel
        ExecutorService executor = Executors.newFixedThreadPool(workerCount);
        List<CompletableFuture<Void>> futures = new ArrayList<>();

        for (int i = 0; i < workerCount; i++) {

```

```
        final int workerId = i;
        CompletableFuture<Void> future = CompletableFuture.runAsync(() ->
{
            processPartitionsForWorker(jobId, workerId);
        }, executor);
        futures.add(future);
    }

    // Step 5: Wait for all workers to complete
    CompletableFuture.allOf(futures.toArray(new
CompletableFuture[0])).join();
    executor.shutdown();

    // Step 6: Finalize job
    JobStats stats = getJobStats(jobId);
    completeJob(jobId, stats.getTotalPartitions(),
        stats.getCompletedPartitions(),
stats.getFailedPartitions());

    log.info("Job {} completed: {} completed, {} failed",
        jobId, stats.getCompletedPartitions(),
stats.getFailedPartitions());

    // Step 7: Record metrics
    metricsService.recordJobMetrics(jobId, stats);

    return jobId;

} catch (Exception e) {
    log.error("Job {} failed with error: {}", jobId, e.getMessage(), e);
    failJob(jobId, e.getMessage());
    throw new RuntimeException("ETL job failed", e);
}

}

private void processPartitionsForWorker(Long jobId, int workerId) {
    log.info("Worker {} started for job {}", workerId, jobId);

    int processedCount = 0;
    while (true) {
        // Claim next partition (thread-safe)
        Long partitionId = partitionService.claimNextPartition(workerId);

        if (partitionId == null) {
            log.info("Worker {}: No more partitions available", workerId);
            break;
        }

        try {
            // Process the partition
            workerService.processPartition(partitionId);
            processedCount++;

            if (processedCount % 10 == 0) {
```

```

        log.info("Worker {} processed {} partitions", workerId,
processedCount);
    }

    } catch (Exception e) {
        log.error("Worker {} failed processing partition {}: {}",
            workerId, partitionId, e.getMessage());
        // Continue with next partition (fault tolerance)
    }
}

log.info("Worker {} completed: {} partitions processed", workerId,
processedCount);
}

private List<String> identifyAppIdsToProcess(String jobType) {
    if ("DAILY".equals(jobType)) {
        // Daily: Only app_ids with new/updated data since last run
        return mccJdbc.queryForList(
            "SELECT DISTINCT app_id FROM adb.source_table " +
            "WHERE last_updated > DATEADD(day, -1, GETDATE())",
            String.class
        );
    } else {
        // Manual/Recovery: All app_ids or specific criteria
        return mccJdbc.queryForList(
            "SELECT DISTINCT app_id FROM adb.source_table",
            String.class
        );
    }
}

private Long createJobRecord(String jobType, String triggeredBy) {
    KeyHolder keyHolder = new GeneratedKeyHolder();
    mccJdbc.update(connection -> {
        PreparedStatement ps = connection.prepareStatement(
            "INSERT INTO mcc.etl_jobs (job_type, started_at, triggered_by,
status) " +
            "VALUES (?, GETDATE(), ?, 'RUNNING')",
            Statement.RETURN_GENERATED_KEYS
        );
        ps.setString(1, jobType);
        ps.setString(2, triggeredBy);
        return ps;
    }, keyHolder);

    return keyHolder.getKey().longValue();
}

private void completeJob(Long jobId, int total, int completed, int failed) {
    mccJdbc.update(
        "UPDATE mcc.etl_jobs SET " +
        "completed_at = GETDATE(), " +
        "total_partitions = ?, " +

```

```

        "completed_partitions = ?, " +
        "failed_partitions = ?, " +
        "status = CASE WHEN ? > 0 THEN 'COMPLETED_WITH_ERRORS' ELSE
'COMPLETED' END " +
        "WHERE job_id = ?",
        total, completed, failed, failed, jobId
    );
}

private void failJob(Long jobId, String errorMessage) {
    mccJdbc.update(
        "UPDATE mcc.etl_jobs SET " +
        "completed_at = GETDATE(), " +
        "status = 'FAILED' " +
        "WHERE job_id = ?",
        jobId
    );
}

private JobStats getJobStats(Long jobId) {
    return mccJdbc.queryForObject(
        "SELECT " +
        "COUNT(*) as total_partitions, " +
        "SUM(CASE WHEN status = 'COMPLETED' THEN 1 ELSE 0 END) as completed, "
+
        "SUM(CASE WHEN status = 'FAILED' THEN 1 ELSE 0 END) as failed, " +
        "SUM(record_count) as total_records " +
        "FROM mcc.etl_partitions WHERE job_id = ?",
        (rs, rowNum) -> new JobStats(
            rs.getInt("total_partitions"),
            rs.getInt("completed"),
            rs.getInt("failed"),
            rs.getLong("total_records")
        ),
        jobId
    );
}
}

@Data
@AllArgsConstructor
class JobStats {
    private int totalPartitions;
    private int completedPartitions;
    private int failedPartitions;
    private long totalRecords;
}

```

## PartitionService.java (Partition Management)

```

@Service
@Slf4j
public class PartitionService {

    @Autowired
    @Qualifier("adbJdbcTemplate")
    private JdbcTemplate adbJdbc;

    @Autowired
    @Qualifier("mccJdbcTemplate")
    private JdbcTemplate mccJdbc;

    @Value("${etl.retry.max-attempts:3}")
    private int maxRetryAttempts;

    public List<Long> createPartitions(Long jobId, List<String> appIds) {
        List<Long> partitionIds = new ArrayList<>();

        for (String appId : appIds) {
            // Count records for this app_id
            Long recordCount = adbJdbc.queryForObject(
                "SELECT COUNT(*) FROM adb.source_table WHERE app_id = ?",
                Long.class, appId
            );

            // Insert partition
            KeyHolder keyHolder = new GeneratedKeyHolder();
            mccJdbc.update(connection -> {
                PreparedStatement ps = connection.prepareStatement(
                    "INSERT INTO mcc.etl_partitions (job_id, app_id, record_count,
status) " +
                    "VALUES (?, ?, ?, 'PENDING')",
                    Statement.RETURN_GENERATED_KEYS
                );
                ps.setLong(1, jobId);
                ps.setString(2, appId);
                ps.setLong(3, recordCount);
                return ps;
            }, keyHolder);

            partitionIds.add(keyHolder.getKey().longValue());
        }

        return partitionIds;
    }

    @Transactional("mccTransactionManager")
    public Long claimNextPartition(int workerId) {
        try {
            // Thread-safe partition claiming using SQL Server UPDLOCK
            List<Long> result = mccJdbc.query(
                "WITH NextPartition AS ( " +
                " SELECT TOP 1 partition_id " +

```

```

        " FROM mcc.etl_partitions WITH (UPDLOCK, ROWLOCK) " +
        " WHERE status = 'PENDING' " +
        "      OR (status = 'FAILED' AND retry_count < ?) " +
        " ORDER BY record_count ASC " + // Process small partitions

first
    ") " +
    "UPDATE p SET " +
    "  status = 'IN_PROGRESS', " +
    "  worker_id = ?, " +
    "  started_at = GETDATE(), " +
    "  retry_count = retry_count + 1 " +
    "OUTPUT INSERTED.partition_id " +
    "FROM mcc.etl_partitions p " +
    "INNER JOIN NextPartition np ON p.partition_id = np.partition_id",
    (rs, rowNum) -> rs.getLong("partition_id"),
    maxRetryAttempts, workerId
    );

    return result.isEmpty() ? null : result.get(0);

} catch (Exception e) {
    log.error("Error claiming partition for worker {}: {}", workerId,
e.getMessage());
    return null;
}
}

public void markPartitionCompleted(Long partitionId, long recordsProcessed,
long processingTimeMs) {
    mccJdbc.update(
        "UPDATE mcc.etl_partitions SET " +
        "status = 'COMPLETED', " +
        "completed_at = GETDATE(), " +
        "record_count = ?, " +
        "processing_time_ms = ? " +
        "WHERE partition_id = ?",
        recordsProcessed, processingTimeMs, partitionId
    );
}

public void markPartitionFailed(Long partitionId, String errorMessage) {
    mccJdbc.update(
        "UPDATE mcc.etl_partitions SET " +
        "status = 'FAILED', " +
        "completed_at = GETDATE(), " +
        "error_message = ? " +
        "WHERE partition_id = ?",
        errorMessage, partitionId
    );
}
}

```



**WorkerService.java (Process Single Partition)**

```
@Service
@Slf4j
public class WorkerService {

    @Autowired
    @Qualifier("adbJdbcTemplate")
    private JdbcTemplate adbJdbc;

    @Autowired
    @Qualifier("mccJdbcTemplate")
    private JdbcTemplate mccJdbc;

    @Autowired
    private PartitionService partitionService;

    @Value("${etl.batch.size:50000}")
    private int batchSize;

    public void processPartition(Long partitionId) {
        long startTime = System.currentTimeMillis();

        try {
            // Get partition details
            Partition partition = getPartitionDetails(partitionId);
            log.info("Processing partition {}: app_id={}, record_count={}",
                    partitionId, partition.getAppId(),
                    partition.getRecordCount());

            // Process in batches
            long totalProcessed = 0;
            long offset = 0;

            while (true) {
                // Fetch batch from ADB
                List<SourceRecord> records = fetchBatchFromAdb(
                    partition.getAppId(), offset, batchSize
                );

                if (records.isEmpty()) {
                    break;
                }

                // Insert/Update into MCC
                int inserted = upsertToMcc(records);
                totalProcessed += inserted;
                offset += batchSize;

                log.debug("Partition {}: Processed {}/{ records",
                        partitionId, totalProcessed, partition.getRecordCount());
            }
        }
```

```

        long processingTime = System.currentTimeMillis() - startTime;
        partitionService.markPartitionCompleted(partitionId, totalProcessed,
processingTime);

        log.info("Partition {} completed: {} records in {}ms",
            partitionId, totalProcessed, processingTime);

    } catch (Exception e) {
        log.error("Partition {} failed: {}", partitionId, e.getMessage(), e);
        partitionService.markPartitionFailed(partitionId, e.getMessage());
        throw new RuntimeException("Partition processing failed", e);
    }
}

private Partition getPartitionDetails(Long partitionId) {
    return mccJdbc.queryForObject(
        "SELECT partition_id, app_id, record_count FROM mcc.etl_partitions
WHERE partition_id = ?",
        (rs, rowNum) -> new Partition(
            rs.getLong("partition_id"),
            rs.getString("app_id"),
            rs.getLong("record_count")
        ),
        partitionId
    );
}

private List<SourceRecord> fetchBatchFromAdb(String appId, long offset, int
limit) {
    String sql =
        "SELECT id, app_id, data_field, last_updated " +
        "FROM adb.source_table " +
        "WHERE app_id = ? " +
        "ORDER BY id " +
        "OFFSET ? ROWS FETCH NEXT ? ROWS ONLY";

    return adbJdbc.query(sql,
        (rs, rowNum) -> new SourceRecord(
            rs.getLong("id"),
            rs.getString("app_id"),
            rs.getString("data_field"),
            rs.getTimestamp("last_updated")
        ),
        appId, offset, limit
    );
}

private int upsertToMcc(List<SourceRecord> records) {
    String sql =
        "MERGE INTO mcc.target_table AS target " +
        "USING (SELECT ? as id, ? as app_id, ? as data_field, ? as
last_updated) AS source " +
        "ON target.id = source.id " +

```

```

        "WHEN MATCHED THEN " +
        "  UPDATE SET app_id = source.app_id, data_field = source.data_field,
last_updated = source.last_updated " +
        "WHEN NOT MATCHED THEN " +
        "  INSERT (id, app_id, data_field, last_updated) " +
        "  VALUES (source.id, source.app_id, source.data_field,
source.last_updated);";

    int[] results = mccJdbc.batchUpdate(sql, new
BatchPreparedStatementSetter() {
        @Override
        public void setValues(PreparedStatement ps, int i) throws SQLException
        {
            SourceRecord record = records.get(i);
            ps.setLong(1, record.getId());
            ps.setString(2, record.getAppId());
            ps.setString(3, record.getDataField());
            ps.setTimestamp(4, record.getLastUpdated());
        }

        @Override
        public int getBatchSize() {
            return records.size();
        }
    });

    return results.length;
}

@Data
@AllArgsConstructor
class Partition {
    private Long partitionId;
    private String appId;
    private Long recordCount;
}

@Data
@AllArgsConstructor
class SourceRecord {
    private Long id;
    private String appId;
    private String dataField;
    private Timestamp lastUpdated;
}

```

## Monitoring & Alerting

### Grafana Dashboard (JSON Config)

```

{
  "dashboard": {
    "title": "ETL Microservice Monitoring",
    "panels": [
      {
        "title": "Job Progress",
        "type": "gauge",
        "targets": [{
          "expr": "(sum(etl_partitions_completed) / sum(etl_partitions_total)) *
100"
        }]
      },
      {
        "title": "Partitions by Status",
        "type": "piechart",
        "targets": [{
          "expr": "sum by (status) (etl_partitions_status)"
        }]
      },
      {
        "title": "Processing Throughput (records/sec)",
        "type": "graph",
        "targets": [{
          "expr": "rate(etl_records_processed_total[5m])"
        }]
      },
      {
        "title": "Worker Activity",
        "type": "graph",
        "targets": [{
          "expr": "etl_active_workers"
        }]
      },
      {
        "title": "Failed Partitions",
        "type": "table",
        "targets": [{
          "expr": "etl_partitions{status='FAILED'}"
        }]
      }
    ]
  }
}

```

## Alerting Rules (PagerDuty/Email)

```

# prometheus-alerts.yml
groups:
- name: etl_alerts
  interval: 1m

```

```
rules:
- alert: EtlJobFailed
  expr: etl_job_status{status="FAILED"} == 1
  for: 1m
  annotations:
    summary: "ETL Job {{ $labels.job_id }} failed"
    description: "Check logs for job_id={{ $labels.job_id }}"

- alert: HighFailureRate
  expr: (sum(etl_partitions_failed) / sum(etl_partitions_total)) > 0.05
  for: 5m
  annotations:
    summary: "ETL failure rate > 5%"
    description: "{{ $value }}% of partitions are failing"

- alert: JobRunningTooLong
  expr: (time() - etl_job_start_time) > 7200 # 2 hours
  for: 5m
  annotations:
    summary: "ETL job running for > 2 hours"
    description: "Job {{ $labels.job_id }} may be stuck"

- alert: NoJobsIn25Hours
  expr: (time() - etl_last_job_time) > 90000 # 25 hours
  for: 5m
  annotations:
    summary: "No ETL jobs in last 25 hours"
    description: "Daily job may have failed to start"
```

---

## Production Incident: Thread Pool Saturation & Downstream Service Overload

### Incident Overview

**Problem:** Microservice experiencing thread pool saturation, high latency, connection pool exhaustion due to slow/unresponsive downstream service calls.

### Symptoms:

- Thread pool 100% utilized (all 200 threads busy)
- API response time increased from 200ms → 30 seconds
- Connection pool wait time > 10 seconds
- HTTP 500 errors for end users
- Database connection pool exhausted

**Root Cause:** Downstream service (external API) responding slowly (15-20 seconds), causing threads to hang waiting for response. No timeouts configured, so threads wait indefinitely.

---

### Immediate Mitigation (Same Day) - Step-by-Step Implementation

## Step 1: Confirm the Issue (15 minutes)

### Check Metrics (Prometheus/Grafana)

```
# Thread pool utilization
http_server_requests_active_threads{application="my-service"} /
http_server_requests_max_threads{application="my-service"} * 100
# Result: 98-100% (saturation)

# API latency (P95)
histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[5m]))
# Result: 30-45 seconds (normally 0.2 seconds)

# Connection pool wait time
hikaricp_connections_pending{pool="downstream-pool"}
# Result: 50+ pending (max pool size: 20)
```

### Check Application Logs

```
# Look for timeout exceptions
grep -i "timeout\|SocketTimeoutException\|ReadTimeoutException" app.log

# Thread dump showing waiting threads
jstack <pid> | grep -A 10 "waiting"
# Output: 150 threads in WAITING state on socket read
```

---

## Step 2: Apply Strict Timeouts (30 minutes)

**Problem:** No timeouts configured → Threads wait indefinitely for downstream service

**Solution:** Configure aggressive timeouts at multiple levels

### A. RestTemplate Timeout Configuration (Spring Boot)

#### Before (No Timeouts)

```
@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate(); // ✗ No timeouts - waits forever
    }
}
```

**After (With Timeouts)**

```

@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate() {
        // Create HTTP client with timeouts
        HttpComponentsClientHttpRequestFactory factory =
            new HttpComponentsClientHttpRequestFactory();

        // Connection timeout: Max time to establish TCP connection
        factory.setConnectTimeout(2000); // 2 seconds

        // Read timeout: Max time waiting for data from server
        factory.setReadTimeout(5000); // 5 seconds

        // Request timeout: Total time for entire request
        factory.setConnectionRequestTimeout(3000); // 3 seconds

        RestTemplate restTemplate = new RestTemplate(factory);

        // Add error handler for fail-fast
        restTemplate.setErrorHandler(new DefaultResponseErrorHandler() {
            @Override
            public void handleError(ClientHttpResponse response) throws
IOException {
                // Log and throw exception immediately
                log.error("Downstream service error: {}",
response.getStatusCode());
                super.handleError(response);
            }
        });

        return restTemplate;
    }
}

```

**B. WebClient Timeout Configuration (Reactive Spring)**

```

@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        HttpClient httpClient = HttpClient.create()
            .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000) // 2 sec
connection timeout
            .responseTimeout(Duration.ofSeconds(5)) // 5 sec read
timeout
    }
}

```

```

        .doOnConnected(conn ->
            conn.addHandlerLast(new ReadTimeoutHandler(5))    // 5 sec idle
timeout
            .addHandlerLast(new WriteTimeoutHandler(5))
        );

        return WebClient.builder()
            .clientConnector(new ReactorClientHttpConnector(httpClient))
            .build();
    }
}

```

### C. Database Connection Pool Timeout

#### application.properties

```

# HikariCP configuration (before: no limits)
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=3000           # 3 sec to get
connection from pool
spring.datasource.hikari.idle-timeout=600000               # 10 min idle before
eviction
spring.datasource.hikari.max-lifetime=1800000             # 30 min max connection
lifetime
spring.datasource.hikari.leak-detection-threshold=60000   # Alert if connection
held >1 min

```

### D. Thread Pool Timeout (Executor Service)

```

@Configuration
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(50);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-worker-");

        // Timeout for queued tasks
        executor.setAwaitTerminationSeconds(30); // Wait 30 sec for tasks to
complete on shutdown
        executor.setWaitForTasksToCompleteOnShutdown(true);

        // Rejection policy: Fail fast when queue full
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
    }
}

```



```
        executor.initialize();
        return executor;
    }
}
```

---

### Step 3: Implement Fail-Fast Behavior (1 hour)

**Problem:** Threads hang waiting for downstream service, blocking other requests

**Solution:** Circuit Breaker pattern using Resilience4j

#### A. Add Resilience4j Dependency

##### pom.xml

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.1.0</version>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

#### B. Configure Circuit Breaker

##### application.yml

```
resilience4j:
  circuitbreaker:
    instances:
      downstreamService:
        # After 5 failures in 10 seconds, open circuit
        failureRateThreshold: 50
        minimumNumberOfCalls: 5
        waitDurationInOpenState: 30s # Stay open for 30 sec before trying again

        # Allow 3 test calls in half-open state
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowSize: 10

        # Timeout configuration
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 3s # Calls >3 sec considered "slow"
```

```

timelimiter:
  instances:
    downstreamService:
      timeoutDuration: 5s # Force timeout after 5 seconds
      cancelRunningFuture: true

bulkhead:
  instances:
    downstreamService:
      maxConcurrentCalls: 10 # Limit concurrent calls
      maxWaitDuration: 1s # Wait max 1 sec for available slot

```

### C. Apply Circuit Breaker to Service Calls

#### DownstreamService.java

```

@Service
@Slf4j
public class DownstreamServiceClient {

    @Autowired
    private RestTemplate restTemplate;

    // Apply circuit breaker + timeout + bulkhead
    @CircuitBreaker(name = "downstreamService", fallbackMethod =
"fallbackResponse")
    @TimeLimiter(name = "downstreamService")
    @Bulkhead(name = "downstreamService")
    public CompletableFuture<String> callDownstreamService(String requestData) {
        log.info("Calling downstream service with data: {}", requestData);

        return CompletableFuture.supplyAsync(() -> {
            try {
                String url = "https://external-api.com/endpoint";
                ResponseEntity<String> response = restTemplate.postForEntity(
                    url, requestData, String.class
                );
                return response.getBody();
            } catch (ResourceAccessException e) {
                // Timeout or connection error
                log.error("Downstream service timeout: {}", e.getMessage());
                throw new RuntimeException("Service unavailable", e);
            }
        });
    }

    // Fallback method when circuit is open or call fails
    public CompletableFuture<String> fallbackResponse(String requestData,
Exception e) {

```

```

        log.warn("Circuit breaker activated, using fallback. Error: {}",
e.getMessage());

        // Return cached response, default value, or error message
        return CompletableFuture.completedFuture(
            "{\"status\": \"fallback\", \"message\": \"Service temporarily
unavailable\"}"
        );
    }
}

```

#### D. Handle Circuit Breaker Events

```

@Component
@Slf4j
public class CircuitBreakerEventListener {

    @Autowired
    private CircuitBreakerRegistry circuitBreakerRegistry;

    @PostConstruct
    public void registerEventListener() {
        CircuitBreaker circuitBreaker =
circuitBreakerRegistry.circuitBreaker("downstreamService");

        circuitBreaker.getEventPublisher()
            .onStateTransition(event -> {
                log.warn("Circuit breaker state changed: {} -> {}",
                    event.getStateTransition().getFromState(),
                    event.getStateTransition().getToState()
                );

                // Send alert when circuit opens
                if (event.getStateTransition().getToState() ==
CircuitBreaker.State.OPEN) {
                    sendAlert("Circuit breaker OPEN: downstream service failing");
                }
            })
            .onError(event -> {
                log.error("Circuit breaker recorded error: {}",
event.getThrowable().getMessage());
            })
            .onSuccess(event -> {
                log.debug("Circuit breaker recorded success, duration: {}ms",
                    event.getElapsedDuration().toMillis());
            });
    }

    private void sendAlert(String message) {
        // Send PagerDuty/email alert
        log.error("ALERT: {}", message);
    }
}

```

```
}
}
```

#### Step 4: Limit Concurrency to Downstream System (30 minutes)

**Problem:** All 200 threads trying to call downstream service simultaneously, overwhelming it

**Solution:** Rate limiting and request throttling

##### A. Bulkhead Pattern (Already configured above)

```
resilience4j:
  bulkhead:
    instances:
      downstreamService:
        maxConcurrentCalls: 10 # Only 10 concurrent calls allowed
        maxWaitDuration: 1s    # If all 10 slots busy, wait max 1 sec
```

##### How it works:

- First 10 requests: Execute immediately
- Request 11-20: Wait up to 1 second for available slot
- Request 21+: Rejected with `BulkheadFullException`

##### B. Rate Limiter (Additional Protection)

```
resilience4j:
  ratelimiter:
    instances:
      downstreamService:
        limitForPeriod: 50 # Max 50 calls
        limitRefreshPeriod: 1s # per 1 second window
        timeoutDuration: 0s # Don't wait, reject immediately
```

```
@Service
public class DownstreamServiceClient {

    @RateLimiter(name = "downstreamService")
    @CircuitBreaker(name = "downstreamService", fallbackMethod =
"fallbackResponse")
    @Bulkhead(name = "downstreamService")
    public CompletableFuture<String> callDownstreamService(String requestData) {
        // ... implementation
    }
}
```

### C. Manual Semaphore Control (Alternative)

```
@Service
public class DownstreamServiceClient {

    // Limit concurrent calls using semaphore
    private final Semaphore concurrencyLimit = new Semaphore(10);

    @Autowired
    private RestTemplate restTemplate;

    public String callDownstreamService(String requestData) {
        boolean acquired = false;
        try {
            // Try to acquire permit (timeout 1 second)
            acquired = concurrencyLimit.tryAcquire(1, TimeUnit.SECONDS);

            if (!acquired) {
                log.warn("Concurrency limit reached, rejecting request");
                throw new ServiceUnavailableException("Too many concurrent
requests");
            }

            // Make the actual call
            return restTemplate.postForObject(
                "https://external-api.com/endpoint",
                requestData,
                String.class
            );

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new RuntimeException("Request interrupted", e);

        } finally {
            if (acquired) {
                concurrencyLimit.release();
            }
        }
    }
}
```

---

## Step 5: Monitoring & Validation (Ongoing)

### A. Expose Circuit Breaker Metrics

**application.properties**

```
# Enable Actuator endpoints
management.endpoints.web.exposure.include=health,metrics,prometheus,circuitbreaker
s
management.endpoint.health.show-details=always

# Circuit breaker health indicator
management.health.circuitbreakers.enabled=true
```

## Check Circuit Breaker Status

```
# Health endpoint
curl http://localhost:8080/actuator/health
# Response:
{
  "status": "UP",
  "components": {
    "circuitBreakers": {
      "status": "UP",
      "details": {
        "downstreamService": {
          "status": "HALF_OPEN",
          "failureRate": "45.5%",
          "slowCallRate": "60.2%"
        }
      }
    }
  }
}

# Metrics endpoint
curl http://localhost:8080/actuator/metrics/resilience4j.circuitbreaker.calls
# Response shows successful, failed, ignored calls

# Circuit breakers endpoint
curl http://localhost:8080/actuator/circuitbreakers
```

## B. Grafana Dashboard Queries

```
# Circuit breaker state (0=CLOSED, 1=OPEN, 2=HALF_OPEN)
resilience4j_circuitbreaker_state{name="downstreamService"}

# Failure rate
rate(resilience4j_circuitbreaker_calls_total{name="downstreamService",kind="failed"}[5m]) /
rate(resilience4j_circuitbreaker_calls_total{name="downstreamService"}[5m]) * 100

# Bulkhead available concurrent calls
resilience4j_bulkhead_available_concurrent_calls{name="downstreamService"}
```

```
# Thread pool active threads
tomcat_threads_current_threads{name="http-nio-8080-exec"} -
tomcat_threads_busy_threads{name="http-nio-8080-exec"}
```

### C. Validate Fix

#### Before Fix:

```
Thread pool utilization: 98-100%
API P95 latency: 30-45 seconds
Error rate: 15%
Downstream calls queued: 50+
```

#### After Fix:

```
Thread pool utilization: 20-30%
API P95 latency: 0.5-2 seconds
Error rate: 2% (only when circuit open)
Downstream calls queued: 0-2
Circuit breaker: CLOSED (healthy)
```

---

## Step-by-Step Deployment Plan

### Phase 1: Apply Timeouts (Low Risk)

```
# Update application.properties with timeouts
# Deploy to 1 instance (canary)
# Monitor for 30 minutes
# If stable, deploy to all instances
```

### Phase 2: Enable Circuit Breaker (Medium Risk)

```
# Add Resilience4j dependency
# Configure circuit breaker with conservative settings:
#   - failureRateThreshold: 80% (high threshold initially)
#   - waitDurationInOpenState: 10s (short wait)
# Deploy to canary
# Monitor circuit breaker state transitions
# Gradually tighten thresholds: 80% → 60% → 50%
```

### Phase 3: Add Bulkhead (High Impact)

```
# Configure bulkhead with maxConcurrentCalls: 20 (start high)
# Deploy to canary
# Monitor rejected calls metric
# Gradually reduce: 20 → 15 → 10
# Find sweet spot where downstream service stable
```

---

## Complete Working Example

### application.yml (Final Configuration)

```
server:
  port: 8080
  tomcat:
    threads:
      max: 200
      min-spare: 10

spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      connection-timeout: 3000
      idle-timeout: 600000
      max-lifetime: 1800000
      leak-detection-threshold: 60000

resilience4j:
  circuitbreaker:
    instances:
      downstreamService:
        failureRateThreshold: 50
        minimumNumberOfCalls: 5
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowSize: 10
        slidingWindowType: COUNT_BASED
        recordExceptions:
          - org.springframework.web.client.ResourceAccessException
          - java.net.SocketTimeoutException
          - java.io.IOException
        ignoreExceptions:
          - java.lang.IllegalArgumentException

timelimiter:
  instances:
    downstreamService:
      timeoutDuration: 5s
```



```

        cancelRunningFuture: true

bulkhead:
  instances:
    downstreamService:
      maxConcurrentCalls: 10
      maxWaitDuration: 1s

ratelimiter:
  instances:
    downstreamService:
      limitForPeriod: 50
      limitRefreshPeriod: 1s
      timeoutDuration: 0s

management:
  endpoints:
    web:
      exposure:
        include: health,metrics,prometheus,circuitbreakers
  endpoint:
    health:
      show-details: always
  health:
    circuitbreakers:
      enabled: true
  metrics:
    export:
      prometheus:
        enabled: true

```

## Complete Service with All Protections

```

@Service
@Slf4j
public class ResilientDownstreamServiceClient {

    @Autowired
    private RestTemplate restTemplate;

    @CircuitBreaker(name = "downstreamService", fallbackMethod =
"fallbackGetData")
    @TimeLimiter(name = "downstreamService")
    @Bulkhead(name = "downstreamService")
    @RateLimiter(name = "downstreamService")
    @Retry(name = "downstreamService")
    public CompletableFuture<ResponseData> getData(String requestId) {
        return CompletableFuture.supplyAsync(() -> {
            log.info("Calling downstream service for requestId: {}", requestId);

            try {
                String url = "https://external-api.com/data/" + requestId;

```

```

        ResponseEntity<ResponseData> response = restTemplate.exchange(
            url,
            HttpMethod.GET,
            null,
            ResponseData.class
        );

        log.info("Downstream service response received: {}",
response.getStatusCode());
        return response.getBody();

    } catch (ResourceAccessException e) {
        log.error("Downstream service timeout for requestId: {}",
requestId);
        throw new ServiceUnavailableException("Downstream service
timeout", e);
    } catch (HttpClientErrorException | HttpServerErrorException e) {
        log.error("Downstream service error: {} - {}", e.getStatusCode(),
e.getMessage());
        throw new ServiceUnavailableException("Downstream service error",
e);
    }
    });
}

// Fallback when circuit breaker opens or call fails
private CompletableFuture<ResponseData> fallbackGetData(String requestId,
Exception e) {
    log.warn("Using fallback for requestId: {}. Reason: {}", requestId,
e.getMessage());

    // Return cached data or default response
    ResponseData fallbackData = getCachedData(requestId)
        .orElse(ResponseData.defaultResponse());

    return CompletableFuture.completedFuture(fallbackData);
}

private Optional<ResponseData> getCachedData(String requestId) {
    // Implement caching logic (Redis, Caffeine, etc.)
    return Optional.empty();
}
}

```

---

## Verification Script

```

#!/bin/bash
# verify-fix.sh - Validate the fix is working

```

```
echo "=== Checking Thread Pool Utilization ==="
curl -s http://localhost:8080/actuator/metrics/tomcat.threads.busy | jq
'.measurements[0].value'

echo "=== Checking Circuit Breaker State ==="
curl -s http://localhost:8080/actuator/health | jq
'.components.circuitBreakers.details'

echo "=== Checking Recent Errors ==="
curl -s http://localhost:8080/actuator/metrics/resilience4j.circuitbreaker.calls | \
jq '.availableTags[] | select(.tag=="kind") | .values'

echo "=== Checking Response Time (P95) ==="
curl -s http://localhost:8080/actuator/metrics/http.server.requests | \
jq '.measurements[] | select(.statistic=="P95") | .value'

echo "=== Checking Connection Pool ==="
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.active | jq
'.measurements[0].value'
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.pending | jq
'.measurements[0].value'
```

---

## Interview Preparation Guide

🔗 STAR Interview Answer: Thread Pool Saturation Incident (Memorize This)

### One-Line Summary for Interviewer:

"I fixed thread starvation caused by slow downstream calls by adding fail-fast timeouts, circuit breaker + bulkhead isolation, and moving long-running work to async processing with comprehensive monitoring."

---

### S — Situation

"In production, one of our critical APIs started timing out during peak traffic. We found the root cause was a downstream dependency becoming very slow (15-20 second response times), and our service was making synchronous calls without strong timeouts. This caused request threads to block and created thread starvation—all 200 threads were stuck waiting for the slow downstream service."

### Key details to mention:

- Production incident during peak hours
  - API response time: 200ms → 30 seconds
  - Thread pool: 98-100% saturation
  - Root cause: Slow downstream service + no timeouts
  - Business impact: Users seeing HTTP 500 errors
-

## T — Task

"My responsibility was to stabilize the system quickly, restore response times, and prevent a single slow dependency from taking down the whole service—without breaking business flows or losing data."

### Key responsibilities:

- Immediate incident response (stabilize within hours)
  - Root cause analysis (why threads saturated)
  - Design resilient solution (prevent recurrence)
  - No business disruption (zero downtime deployment)
- 

## A — Action

"I handled it in three layers: **immediate mitigation, resilient design, and long-term prevention.**"

### 1) Immediate Mitigation (Same Day)

**Diagnosed the Problem (15 minutes):** "I confirmed the issue using metrics and logs: thread pool saturation (98%), increased latency (P95: 30 seconds), and connection pool wait time (50+ pending connections)."

### Evidence gathered:

- Prometheus metrics showing thread pool exhaustion
- Application logs showing `SocketTimeoutException`
- Thread dumps showing 150 threads in WAITING state

**Applied Strict Timeouts (30 minutes):** "I applied strict timeouts for downstream calls and reduced the maximum wait time so threads fail fast instead of hanging."

### Specific changes:

```
// Added to RestTemplate configuration
factory.setConnectTimeout(2000);      // 2 seconds to establish connection
factory.setReadTimeout(5000);        // 5 seconds to receive response
factory.setConnectionRequestTimeout(3000); // 3 seconds to get connection from
pool
```

### Why this worked:

- Threads no longer wait indefinitely
- Failed requests return in 5 seconds instead of hanging
- Thread pool freed up for new requests

**Limited Concurrency (30 minutes):** "I temporarily limited concurrency to the downstream system to stop overload and protect the rest of the service."

### Implementation:

```
// Added Semaphore to limit concurrent calls
private final Semaphore concurrencyLimit = new Semaphore(10);

// Only 10 threads can call downstream at once
// 11th request either waits 1 second or gets rejected
```

**Result:** Within 2 hours, thread pool utilization dropped from 98% → 30%, API latency from 30s → 2s.

---

## 2) Resilience Improvements (Next Release - 2-3 Days)

**Circuit Breaker Pattern:** "I implemented a circuit breaker using Resilience4j so when downstream was unhealthy, we returned a controlled fallback response instead of waiting."

### Configuration:

```
resilience4j:
  circuitbreaker:
    failureRateThreshold: 50           # Open circuit after 50% failures
    waitDurationInOpenState: 30s      # Stay open for 30 seconds
    minimumNumberOfCalls: 5           # Need 5 calls to calculate failure rate
```

### How it works:

1. After 5 failures in 10 requests → Circuit OPEN
2. For next 30 seconds, all requests fail immediately with fallback response
3. After 30 seconds, allow 3 test requests (HALF\_OPEN state)
4. If test succeeds → Circuit CLOSED (back to normal)

### Business benefit:

- Fast failure (< 1 second) instead of waiting 5 seconds
- Gives downstream service time to recover
- Users get fallback response (cached data) instead of error

**Bulkhead Isolation:** "I added bulkhead isolation: separate thread pool for downstream calls, so it couldn't exhaust the main request threads."

### Implementation:

```
resilience4j:
  bulkhead:
    maxConcurrentCalls: 10           # Only 10 concurrent downstream calls
    maxWaitDuration: 1s              # Wait max 1 second for available slot
```

### Result:

- Downstream issues isolated to 10 threads max
- Remaining 190 threads handle other requests normally
- System degrades gracefully instead of total failure

**Async Processing with Kafka:** "For non-critical flows, I moved to async processing using Kafka, so user requests were acknowledged quickly and work completed in the background."

#### Example flow:

Before (Synchronous):

User Request → API → Downstream Service (15 sec) → Response

Total: 15 seconds, thread blocked entire time

After (Asynchronous):

User Request → API → Publish to Kafka → 202 Accepted (< 100ms)

Background Worker → Consume from Kafka → Call Downstream → Update DB

Total user wait: 100ms, work happens in background

#### Implementation:

```
@PostMapping("/process")
public ResponseEntity<String> processRequest(@RequestBody Request req) {
    // Publish to Kafka immediately
    kafkaTemplate.send("background-tasks", req);

    // Return 202 Accepted to user
    return ResponseEntity.accepted()
        .body("Request queued for processing");
}
```

**Smart Retry Strategy:** "I used retries only for retryable failures, with exponential backoff + jitter to avoid retry storms."

#### Configuration:

```
resilience4j:
  retry:
    maxAttempts: 3
    waitDuration: 1s # First retry after 1 second
    enableExponentialBackoff: true
    exponentialBackoffMultiplier: 2 # 1s, 2s, 4s
    enableRandomizedWait: true # Add jitter (±20%)
    retryExceptions:
      - java.net.SocketTimeoutException # Retry on timeout
      - java.io.IOException # Retry on network error
    ignoreExceptions:
      - java.lang.IllegalArgumentException # Don't retry bad request
```

### Why exponential backoff + jitter:

- Prevents "thundering herd" (all retries at same time)
  - Gives downstream service breathing room to recover
  - Random jitter spreads retry load over time
- 

### 3) Prevention & Observability

**Dashboards and Alerts:** "I added dashboards and alerts for thread pool utilization, downstream latency, circuit breaker state, and error rates."

#### Grafana Dashboard Panels:

1. Thread Pool Utilization: Real-time % busy threads
2. API Latency (P50, P95, P99): Response time percentiles
3. Circuit Breaker State: CLOSED/OPEN/HALF\_OPEN indicator
4. Downstream Call Success Rate: % successful calls
5. Bulkhead Available Slots: How many concurrent calls allowed
6. Connection Pool Metrics: Active/idle/pending connections

#### PagerDuty Alerts:

- Thread pool > 80% for 5 minutes → P2 alert
- Circuit breaker OPEN → P1 alert (immediate)
- Downstream error rate > 10% → P2 alert
- API P95 latency > 5 seconds → P3 alert

**SLOs and Error Budgets:** "We introduced SLOs (Service Level Objectives) and error budgets for the dependency, and documented runbooks for on-call response."

#### SLO Example:

```
Service: Critical API
SLI (Indicator): P95 latency < 2 seconds
SLO (Objective): 99.5% of requests meet SLI (30-day window)
Error Budget: 0.5% = 216 minutes of P95 > 2s per month
```

```
If error budget exhausted:
→ Freeze new features
→ Focus on reliability improvements
→ Investigate root causes
```

#### Runbook Created:

```
# Runbook: Downstream Service Slow Response

## Symptoms
- Thread pool > 80% utilization
```

```
- API latency > 5 seconds
- Circuit breaker OPEN

## Immediate Actions
1. Check downstream service status page
2. Verify circuit breaker is working (fast failures)
3. Check bulkhead slots (should be limited to 10)
4. Review recent deployments (ours or downstream)

## Mitigation
- If downstream issue: Wait for their team to fix
- If our issue: Reduce bulkhead limit (10 → 5)
- If critical: Enable manual fallback (cached data)

## Post-Incident
- Review circuit breaker thresholds
- Analyze retry patterns
- Update timeout values if needed
```

R — Result

"We reduced API timeouts significantly, stabilized peak traffic, prevented thread starvation, and made the system resilient even when the downstream was slow. After that, the service stayed stable in subsequent spikes, and production incidents dropped."

Quantifiable Results:

| Metric                        | Before Fix         | After Fix               | Improvement    |
|-------------------------------|--------------------|-------------------------|----------------|
| Thread Pool Utilization       | 98-100%            | 20-30%                  | 70% reduction  |
| API P95 Latency               | 30-45 seconds      | 0.5-2 seconds           | 15x faster     |
| Error Rate (during incidents) | 15%                | 2%                      | 87% reduction  |
| Incident Frequency            | 2-3 per month      | 0 in 3 months           | Zero incidents |
| MTTR (Mean Time To Recovery)  | 4-6 hours (manual) | 2-5 minutes (automatic) | 48x faster     |
| Downstream Call Queue         | 50+ pending        | 0-2 pending             | 96% reduction  |

Business Impact:

- **Zero downtime** during deployment (rolling update with circuit breaker)
- **Customer satisfaction** improved: No more timeout errors
- **Engineering velocity** improved: Team confident in resilience
- **Cost savings:** Didn't need to over-provision servers (200 → 50 threads sufficient)

Technical Wins:

- System handles 5x peak traffic without degradation
- Downstream outages don't cascade to our service



- Clear observability: Know exactly when/why things fail
  - Incident response time: 6 hours → 5 minutes (automated recovery)
- 

## 💧 Finish Strong: Key Lesson

**"The key lesson: Never let one slow dependency consume all your threads—fail fast and isolate it."**

### Three principles I learned:

#### 1. Fail Fast, Fail Loud

- Set aggressive timeouts (2-5 seconds)
- Don't let threads hang indefinitely
- Return errors quickly so users can retry

#### 2. Isolate Failures

- Circuit breaker: Stop calling broken service
- Bulkhead: Limit blast radius to dedicated thread pool
- Async processing: Decouple user request from slow work

#### 3. Observe Everything

- You can't fix what you can't see
  - Metrics → Dashboards → Alerts → Runbooks
  - SLOs enforce discipline and prioritization
- 

## Alternative Stories (If Asked for Different Examples)

### Story 2: Kafka Consumer Lag Spike

**Quick version:** "We had Kafka consumer lag spike to 10 million messages during a database outage. I implemented retry with exponential backoff, moved failed messages to DLQ (Dead Letter Queue), and added consumer scaling based on lag metrics. Reduced lag recovery time from 4 hours to 30 minutes."

### Story 3: Database Connection Pool Exhaustion

**Quick version:** "Our database connection pool was exhausted during high traffic. I identified long-running transactions holding connections for minutes. Added statement timeouts (30 seconds), reduced connection max lifetime, and implemented connection leak detection. Pool utilization dropped from 100% to 40%."

---

## Practice Questions & Crisp Answers

**Q: "Why didn't you just increase thread pool size to 500?"**

**Answer:** "That's treating the symptom, not the cause. More threads = more concurrent calls to the already-slow downstream service, making it even slower. Plus, 500 threads consume 500MB+ memory just for stacks. The real fix was isolating the downstream calls with bulkhead (10 threads max) and failing fast with timeouts."

**Q: "What if the fallback response isn't acceptable to users?"**

**Answer:** "We prioritized the fallback strategy:

1. **Cached data** from Redis (best experience, slightly stale)
2. **Default safe value** (e.g., show 'unavailable' instead of error)
3. **Graceful error message** (tell user to retry in 30 seconds)

For critical flows where fallback isn't acceptable, we used **Kafka async processing**: Accept the request (202), process later when downstream recovers."

**Q: "How do you test circuit breaker behavior?"**

**Answer:** "Three ways:

1. **Chaos engineering**: Use Netflix Chaos Monkey to inject downstream failures in staging
2. **Load testing**: JMeter scripts that simulate 1000 req/sec, then kill downstream service, verify circuit opens
3. **Manual testing**: Deploy to staging, make downstream return 500 errors, watch circuit breaker open in Grafana

We validated: Circuit opens after 5 failures, stays open for 30 seconds, then allows test calls."

**Q: "What was the hardest part of this incident?"**

**Answer:** "The hardest part was **diagnosing under pressure**. With 98% thread utilization, even SSH into the server was slow. I had to use thread dumps and metrics to identify the problem (waiting on socket read) instead of blindly restarting.

Second hardest: **Convincing stakeholders** to move to async processing. They wanted instant responses. I ran A/B test: 202 Accepted (100ms) vs Synchronous (15 seconds). 202 approach had 10x higher user satisfaction."

---

### 30-Second Elevator Pitch Version

**If interviewer says: "Tell me about a challenging production incident you resolved"**

**Your answer:** "Our API was timing out because a slow downstream service consumed all 200 request threads. I fixed it same-day by adding strict timeouts (5 seconds) and concurrency limits (10 max concurrent calls). Then I added Resilience4j circuit breaker to fail fast when downstream was unhealthy, bulkhead isolation to prevent thread starvation, and moved non-critical work to Kafka async processing. Result: Latency dropped from 30 seconds to 2 seconds, zero incidents in 3 months. The key lesson: never let one slow dependency consume all your threads—fail fast and isolate it."

---

## STAR Method: ETL Crash & Recovery (Main Challenge Story)

**Situation:** "We had a critical ETL job crash while processing data for 3,500 app IDs from our ADB database to MCC database. The job was trying to process millions of records in a single run and failed due to memory exhaustion. Business operations were blocked because critical data wasn't available in the target system."

**Task:** "My responsibility was to:

1. Immediately recover the failed data transfer (get business unblocked within 4 hours)
2. Design a long-term solution to prevent similar failures
3. Implement monitoring to detect issues early
4. Ensure the solution could handle both normal daily volumes (low) and spike scenarios (3,500+ apps)"

**Action:** "I implemented a three-phase approach:

### **Phase 1: Emergency Recovery (Day 1)**

- Created a partition tracking table to split 3,500 app\_ids into independent units of work
- Wrote a PowerShell script with 10 parallel workers to process partitions concurrently
- Each worker claimed app\_ids using thread-safe SQL (UPDLOCK) to avoid conflicts
- Processed small app\_ids first for quick wins, showing immediate progress to stakeholders
- **Result:** Recovered all data in 3 hours

### **Phase 2: Permanent Solution (Day 2-3)**

- Built Spring Boot microservice with partition-based architecture
- Implemented CompletableFuture for 10 parallel workers
- Added fault tolerance: Failed partitions don't block others, auto-retry up to 3 times
- Made it idempotent: Safe to rerun without creating duplicates (using MERGE statements)
- Scheduled daily at 2 AM via @Scheduled cron, processes only changed app\_ids (incremental)

### **Phase 3: Monitoring & Alerting (Day 4)**

- Integrated Prometheus metrics (job progress, failure rate, throughput)
- Built Grafana dashboard for real-time visibility
- Configured PagerDuty alerts:
  - Job failure
  - Failure rate > 5%
  - Processing time > 2 hours
  - No job execution in 25 hours (scheduler health)"

**Result:** "Successfully delivered:

- **Zero downtime** since implementation (3 months in production)
- **Normal daily runs:** 5-15 minutes for typical load (10-100 apps)
- **Spike handling:** Processed 3,500 apps in 2.5 hours during month-end load test
- **Resilience:** 99.8% partition success rate, failed partitions auto-retry and complete
- **Visibility:** Team can see real-time progress, no more 'job is running, not sure when it'll finish'
- **Business impact:** Reduced data latency from 24+ hours (after crash) to < 1 hour, enabled real-time reporting"

## Technical Deep-Dive Questions

**Q1: "Why did you partition by app\_id instead of just increasing memory?"**

**Answer:** "Three reasons:

1. **Scalability:** Some app\_ids have 100K+ records, others have <1K. Processing all at once means one giant transaction. Partitioning allows independent processing of small and large app\_ids in parallel.
2. **Fault tolerance:** If one app\_id fails (corrupt data, constraint violation), it doesn't block other 3,499 app\_ids. We mark it FAILED and continue. With monolithic approach, one failure kills entire job.
3. **Resource efficiency:** Instead of allocating 64GB RAM to hold all records, we process 50K records at a time per worker. Total memory: ~2-4GB. More cost-effective."

**Q2: "How do you ensure exactly-once processing? What if the job crashes mid-run?"**

**Answer:** "Multi-layered approach:

**Layer 1: Partition-level tracking**

- Each app\_id has status: PENDING → IN\_PROGRESS → COMPLETED
- If job crashes, IN\_PROGRESS partitions are re-claimable (treated like PENDING)
- Completed partitions are never reprocessed

**Layer 2: Database-level idempotency**

- Use MERGE statement (upsert): INSERT if not exists, UPDATE if exists
- Based on primary key (id), so duplicate processing just overwrites with same data
- No duplicate records in target

**Layer 3: Transaction boundaries**

- Each batch (50K records) is one transaction
- If batch fails, it rolls back, partition marked FAILED
- Next retry starts from beginning of that partition

**Example crash scenario:**

- Processing 3,500 app\_ids
- Crash after 2,000 completed
- On restart: Claim remaining 1,500 PENDING + retry any FAILED partitions
- No data loss, no duplicates"

**Q3: "How do you handle variable partition sizes (some app\_ids have 100K records, others 1K)?"**

**Answer:** "Two strategies:

**Strategy 1: Priority ordering**

- Query: `ORDER BY record_count ASC`
- Process small partitions first (quick wins)
- Large partitions distributed across workers naturally
- Benefit: Shows progress quickly (90% of partitions done in 30 min), remaining 10% (large) finish gradually

**Strategy 2: Batch processing within partition**

- Even if app\_id has 100K records, fetch in batches of 50K
- Use SQL OFFSET/FETCH: `OFFSET 0 ROWS FETCH NEXT 50000 ROWS ONLY`
- Process batch, commit, fetch next batch
- Memory stays constant regardless of partition size

**Result:** Worker never holds more than 50K records in memory, even for million-record app\_ids"

#### **Q4: "What happens if MCC database goes down during processing?"**

**Answer:** "Graceful degradation:

##### **Scenario 1: Short outage (< 5 minutes)**

- Worker's UPSERT statement times out after connection-timeout (30 seconds)
- SQLException caught, partition marked FAILED
- Other workers continue with their partitions
- Failed partition retries automatically (up to 3 times)
- If MCC comes back up, retry succeeds

##### **Scenario 2: Extended outage (> 1 hour)**

- Job eventually completes with some partitions FAILED
- Manual trigger later: `POST /api/etl/reset-failed`
- Re-claims only FAILED partitions, processes them
- No need to reprocess 3,500 partitions, just the failed ones

##### **Monitoring alert:**

- If failure rate > 5%, PagerDuty alert fires
- On-call engineer investigates: 'Oh, MCC database is down'
- Decision: Wait for MCC to recover, then `POST /api/etl/reset-failed`"

#### **Q5: "How do you monitor this in production?"**

**Answer:** "Three levels:

##### **Level 1: Real-time dashboard (Grafana)**

- Job progress gauge: 73% complete
- Partition status pie chart: 2500 completed, 200 in-progress, 50 failed, 750 pending
- Throughput graph: 150K records/sec
- Worker activity: 10 workers active

##### **Level 2: Automated alerts (PagerDuty)**

- Job failure: Immediate alert
- High failure rate (>5%): Alert after 5 minutes
- Job stuck (>2 hours): Alert
- Missed daily run: Alert if no job in 25 hours

##### **Level 3: REST APIs for on-demand checks**

```
# Overall status
curl http://etl-service:8080/api/etl/status
# Response: {"jobId": 123, "progress": 73.5, "status": "RUNNING"}

# Partition details
curl http://etl-service:8080/api/etl/partitions?status=FAILED
# Response: [{"appId": "APP-5678", "error": "Connection timeout"}]
```

### Historical analysis:

- Query etl\_jobs table: Average duration, failure patterns
- Query etl\_metrics: Throughput trends over time"

### Behavioral Questions

#### "Tell me about a time you had to work under pressure with tight deadlines"

**Answer:** "During the ETL crash incident, business was unable to generate critical month-end reports. VP asked: 'How fast can you fix this?'"

#### Pressure points:

- 3,500 app\_ids stuck, millions of records
- Business deadline: 4 PM same day (6 hours away)
- No existing recovery mechanism

#### My approach:

1. **Triaged immediately (15 min):** Checked MCC database, found only 200/3,500 apps completed
2. **Quick win first (2 hours):** PowerShell script with 10 workers, got data flowing
3. **Communicated progress (every 30 min):** 'Recovered 500 apps, 2,500 to go, on track for 3 PM'
4. **Delivered early:** Completed at 2:45 PM, 1 hour before deadline

**Key lesson:** Under pressure, focus on **quick wins** (PowerShell script) before **perfect solution** (Spring Boot microservice). Stakeholders care about results, not elegance."

#### "How do you ensure code quality while moving fast?"

**Answer:** "Balance speed with maintainability:

##### Day 1 (Emergency): PowerShell script

- Quick and dirty, but **tested incrementally**
- Ran with 100 test app\_ids first, verified target data
- Then scaled to 3,500

##### Day 2-4 (Permanent solution): Spring Boot

- **Code review:** Had senior engineer review partition claiming SQL (critical for thread safety)
- **Unit tests:** WorkerService batch processing logic

- **Integration test:** End-to-end with 100 test app\_ids in staging
- **Load test:** 3,500 app\_ids in pre-prod, measured 2.5 hour duration

**Production rollout:**

- Deployed Friday evening (low-traffic window)
- Monitored first run Saturday 2 AM
- On-call over weekend (no issues)

**Result:** Zero production incidents in 3 months, because we **tested incrementally** at each step"

---

## Next Steps: Let's Build This

Ready to implement? Here's the plan:

**Phase 1: Database Setup** (You'll do this)

- Run SQL scripts to create `etl_partitions`, `etl_jobs` tables
- Insert test data (100 app\_ids) in ADB

**Phase 2: Spring Boot Code** (I'll generate)

- Complete CoordinatorService implementation
- WorkerService with batch processing
- REST controllers for monitoring

**Phase 3: Testing** (We'll do together)

- Unit test: Partition claiming logic
- Integration test: End-to-end with 100 app\_ids
- Load test: 3,500 app\_ids

**Phase 4: Deployment** (I'll provide instructions)

- Build JAR, deploy to server
- Configure Windows Service / Linux systemd
- Setup Grafana dashboard

**Ready to start coding? Let me know and I'll generate the complete Spring Boot application!**